

Customized pugin to support typescript and playwright automation:

Used Ask mode to understand the framework and identify the files to be updated for supporting playwright and typescript

PROMPT:

ASK MODE: Read the Entire workspace and understand how I implemented the selenium Java

I want to add the playwright typescript

OUTPUT:

```
graph TD
    subgraph UI_Layer [UI Layer (panel.html)]
        direction TB
        L1["- Language: Java, C#, Python, TypeScript"]
        L2["- Engine: Selenium, Playwright, Cypress, Puppeteer"]
    end

    subgraph Chat_Logic [Chat Logic (chat.js)]
        direction TB
        L3["- getPromptKeys() → Maps language + engine to prompt keys"]
        L4["- Currently: java + selenium → SELENIUM_JAVA_PAGE_ONLY"]
        L5["- Other combos → \"unsupported\" message"]
    end

    subgraph Prompt_Templates [Prompt Templates (prompts.js)]
        direction TB
        L6["- generatePageObjectModel() → Generates Page Object Model class in Java"]
    end

    UI_Layer --> Chat_Logic
    Chat_Logic --> Prompt_Templates
```

The diagram illustrates the architecture of a web browser automation system, organized into three main layers:

- UI Layer (panel.html)**
 - Language: Java, C#, Python, TypeScript
 - Engine: Selenium, Playwright, Cypress, Puppeteer
- Chat Logic (chat.js)**
 - `getPromptKeys()` → Maps language + engine to prompt keys
 - Currently: java + selenium → `SELENIUM_JAVA_PAGE_ONLY`
 - Other combos → "unsupported" message
- Prompt Templates (prompts.js)**
 - `generatePageObjectModel()` → Generates Page Object Model class in Java

The flow of data and control is as follows:

- The **UI Layer** interacts with the **Chat Logic** layer.
- The **Chat Logic** layer interacts with the **Prompt Templates** layer.

	- Imports: org.openqa.selenium.*	
	- Includes WebDriver setup, explicit waits	
	- JavaDoc comments	
	CUCUMBER_WITH_SELENIUM_JAVA_STEPS	
	- Generates: Gherkin feature + Java step defs	
	- Includes full WebDriver initialization	
	LLM APIs (groq-api.js, openai-api.js, testleaf-api.js)	
	- Sends prompts + DOM content to LLM	
	- Returns generated code	
	Display (Prism syntax highlighting)	
	- JavaScript, Gherkin, Java already supported	
	- TypeScript support available (prism-typescript.js)	

File Summary

File	Purpose	Selenium Java Content
------	---------	-----------------------

File	Purpose	Selenium Java Content
src/scripts/prompts.js	Prompt templates	SELENIUM_JAVA_PAGE_ONLY, CUCUMBER_WITH_SELENIUM_JAVA_STEPS
src/scripts/chat.js	Chat logic & prompt selection	getPromptKeys(), isJavaSelenium()
panel.html	UI selectors	Language/Engine dropdowns + Mode checkboxes
src/scripts/api/	LLM integrations	Uses selected model/provider

In Agent mode -

I started asking to update each file step by step to accommodate typescript and playwright capabilities

Step1: Go to prompts.js and add new prompt templates for Playwright TypeScript

Take selenium Java as reference

PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY

PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER

CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS

Summary of Changes

1. PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY

- Generates TypeScript Page Object Model class for Playwright
- Uses Playwright v1.40+ imports (Page, Locator)
- Includes JSDoc comments
- Returns exported TypeScript class

2. PLAYWRIGHT_TYPESCRIPT_WITH_CUCUMBER

- Generates both Gherkin feature file AND TypeScript Page Object
- Feature file uses Scenario Outline with Examples
- Page Object includes methods for all DOM elements
- Strongly-typed TypeScript output

3. CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS

- Generates both Gherkin feature file AND TypeScript step definitions
- Step definitions use `@cucumber/cucumber` decorators
- Full Playwright setup: browser launch/close, page management
- Async-first approach with proper waits (`waitFor`)
- Strongly-typed parameters and functions

4. Updated CODE_GENERATOR_TYPES

Added all three new types as constants for consistency across the codebase.

STEP 2 performed to update the chat.js

Go to chat.js

Update getPromptKeys() method to handle Playwright/TypeScript combinations

Prompt Mapping Summary

Selection	Java/Selenium	TypeScript/Playwright
Feature Only	CUCUMBER_ONLY	CUCUMBER_ONLY
Page Only	SELENIUM_JAVA_PAGE_ONLY	PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
Feature + Page	CUCUMBER_WITH_SELENIUM_JAVA_STEPS	CUCUMBER_WITH_PLAYWRIGHT_TYPESCRIPT_STEPS
None Selected (Default)	SELENIUM_JAVA_PAGE_ONLY	PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY

STEP3- Go to panel.html

Add the logics to handle playwright typescript same like selenium Java

Changes to panel.html:

1. Renamed Checkboxes (Line ~32)

- Changed from: `name="javaGenerationMode"`
- Changed to: `name="generationMode"` (generic for all language/engine combinations)
- Added container ID: `generationOptionsContainer` for easier manipulation

2. Added Dynamic Logic (Bottom of file)

Added an inline JavaScript function `initializeGenerationModeLogic()` that:

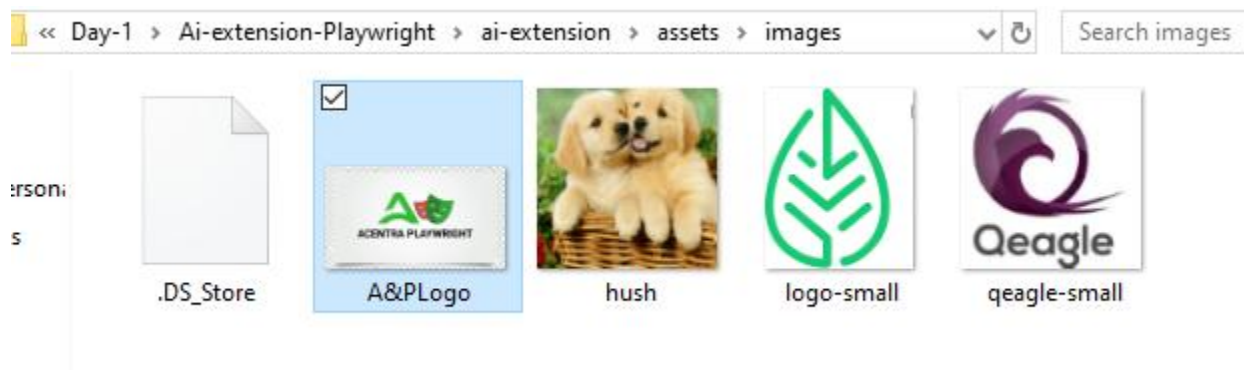
Shows generation mode checkboxes ONLY for supported combinations:

- ☒ Java + Selenium
- ☒ TypeScript + Playwright

Hides for unsupported combinations:

- ☒ Java + Playwright
- ☒ TypeScript + Selenium
- ☒ Other combinations

After the required changes done to add Playwright capabilities, I have added a logo and model as well



Updated manifest.json file

```

    },
    "action": {
      "default_title": "Extension for AI Code Generation",
      "default_icon": {
        "16": "assets/images/A&PLogo.png",
        "32": "assets/images/A&PLogo.png",
        "48": "assets/images/A&PLogo.png",
        "128": "assets/images/A&PLogo.png"
      }
    },
    "icons": {
      "16": "assets/images/A&PLogo.png",
      "32": "assets/images/A&PLogo.png",
      "48": "assets/images/A&PLogo.png",
      "128": "assets/images/A&PLogo.png"
    },
    "side_panel": {

```

And appConfig file

```

ai-extension > src > config > JS appConfig.js > ...
1  export const appConfig = {
2    appName: 'AI Code Generator',
3    logoPath: 'assets/images/A&PLogo.png',
4    colors: {
5      primary: '#ff6b2b',
6      primaryLight: '#234067',
7      accent: 'rgb(43, 116, 142)',
8      success: '#920c2e',
9      successDark: '#a04549',
10     danger: '#ff0000',
11     darkBg: '#2a2a2a',
12     panelBg: '#3a3a3a',
13     mutedText: '#850e1a',
14     // Inspector toggle specific colors
15     gradientStart: '#275fa7',
16     shadowColor: 'rgba(255,107,43,0.3)',
17     shadowColorHover: 'rgba(255,107,43,0.4)',
18     shadowColorActive: 'rgba(255,107,43,0.1)',
19     borderWhite: 'rgba(255,255,255,0.1)'
20   }
21 };

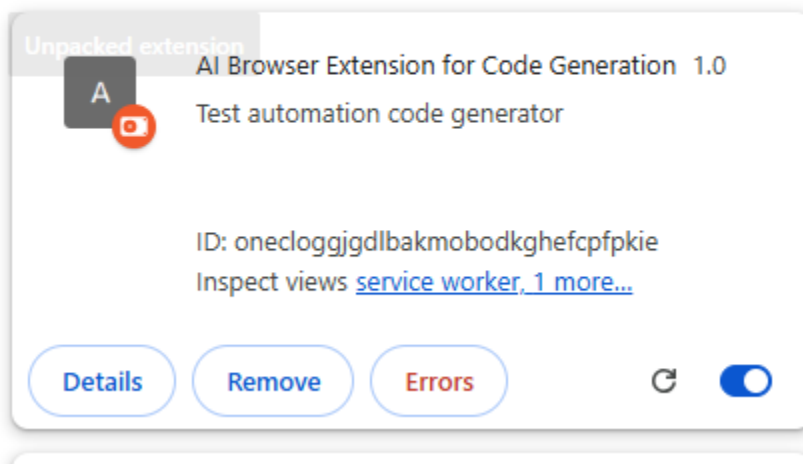
```

Model added:

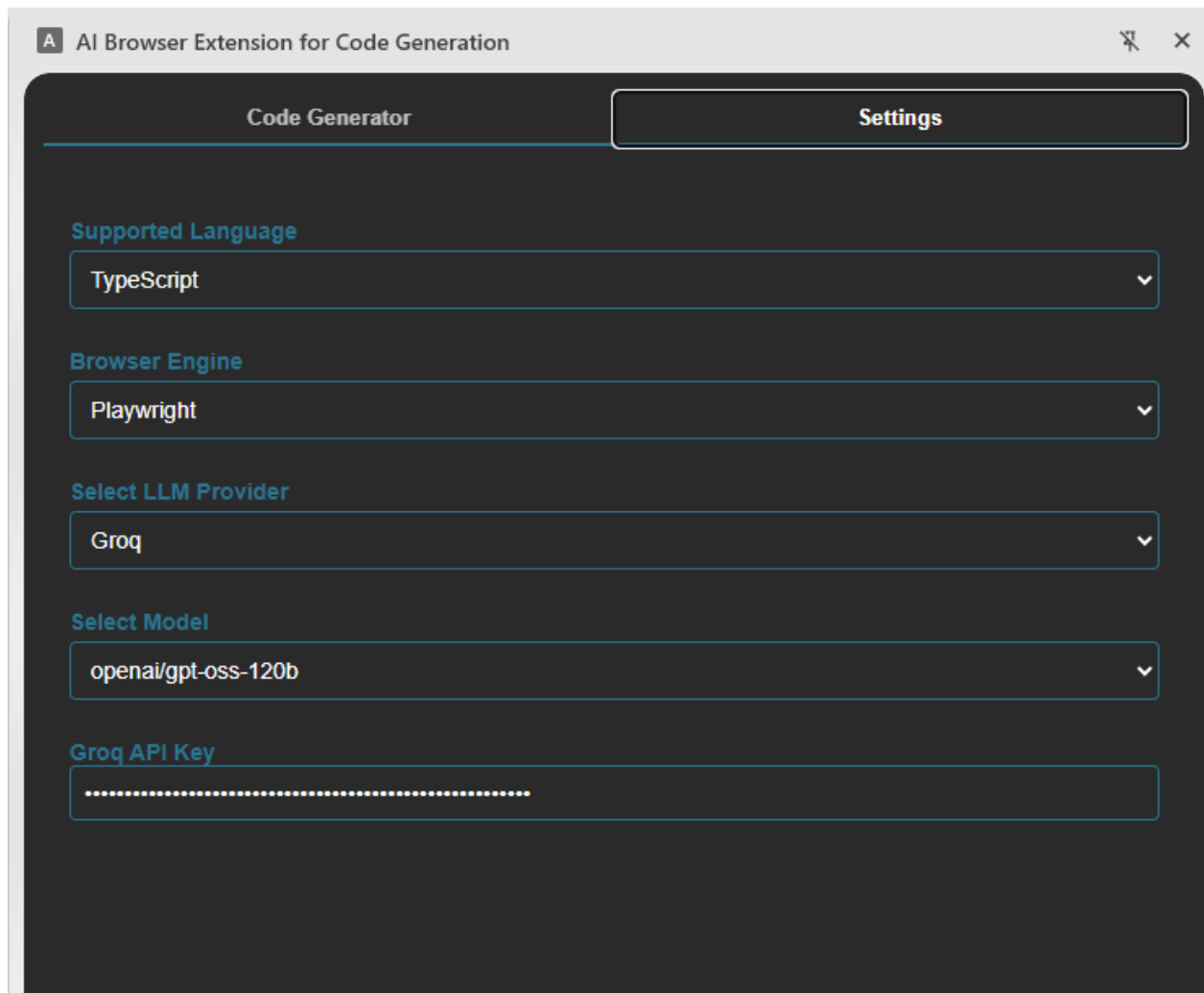
```
ai-extension > src > scripts > JS popup.js > document.addEventListener('DOMContentLoaded') callback > modelsByProvider > openai
5  document.addEventListener('DOMContentLoaded', async () => {
91  },
92
93  // Initialize storage access
94  const storage = chrome.storage?.sync || chrome.storage?.local;
95  if (!storage) {
96    console.error('Chrome storage is not available');
97    return;
98  }
99
100  const providerSelect = document.getElementById('providerSelect');
101  const modelSelect = document.getElementById('modelSelect');
102  const groqApiKeyInput = document.getElementById('groqApiKey');
103  const openaiApiKeyInput = document.getElementById('openaiApiKey');
104  const testleafApiKeyInput = document.getElementById('testleafApiKey');
105
106  // Model options by provider
107  const modelsByProvider = {
108    groq: [
109      { value: 'deepseek-r1-distill-llama-70b', label: 'deepseek-r1-distill-llama-70b' },
110      { value: 'llama-3.3-70b-versatile', label: 'llama-3.3-70b-versatile' },
111      { value: 'openai/gpt-oss-120b', label: 'openai/gpt-oss-120b' },
112    ],
113    openai: [
114
```

Unpacked the utility after saving the changes in VS Code

All Extensions



Supported language and browser engine has been added. Copied the Groq key and added in the code.



Generated a feature file for salesforce login page

Feature: Salesforce login functionality

Scenario Outline: Successful login with valid credentials

Given I open the Salesforce login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I click the Login button

Then I should be redirected to the Salesforce home page

Examples:

username	password	
"ram.kumar@mysalesforce.in"	"R@mkumar123"	
"sita.rao@mysalesforce.in"	"S1t@R0#2023"	
"kumar.patel@mysalesforce.in"	"Patel!456"	

Scenario Outline: Login with Remember me option selected

Given I open the Salesforce login page

When I type "<username>" into the Username field

And I type "<password>" into the Password field

And I check the Remember me checkbox

And I click the Login button

Then I should see the username pre-filled on the next visit

Examples:

username	password	
"lakshmi.nair@mysalesforce.in"	"Lakshmi#789"	
"arun.murthy@mysalesforce.in"	"Arun@2024"	

Scenario: Access Forgot Password page

Given I open the Salesforce login page

When I click the Forgot Your Password link

Then I should be on the password recovery page

Scenario Outline: Use custom domain login

Given I open the Salesforce login page

When I click the Use Custom Domain link

And I type "<customDomain>" into the Custom Domain field

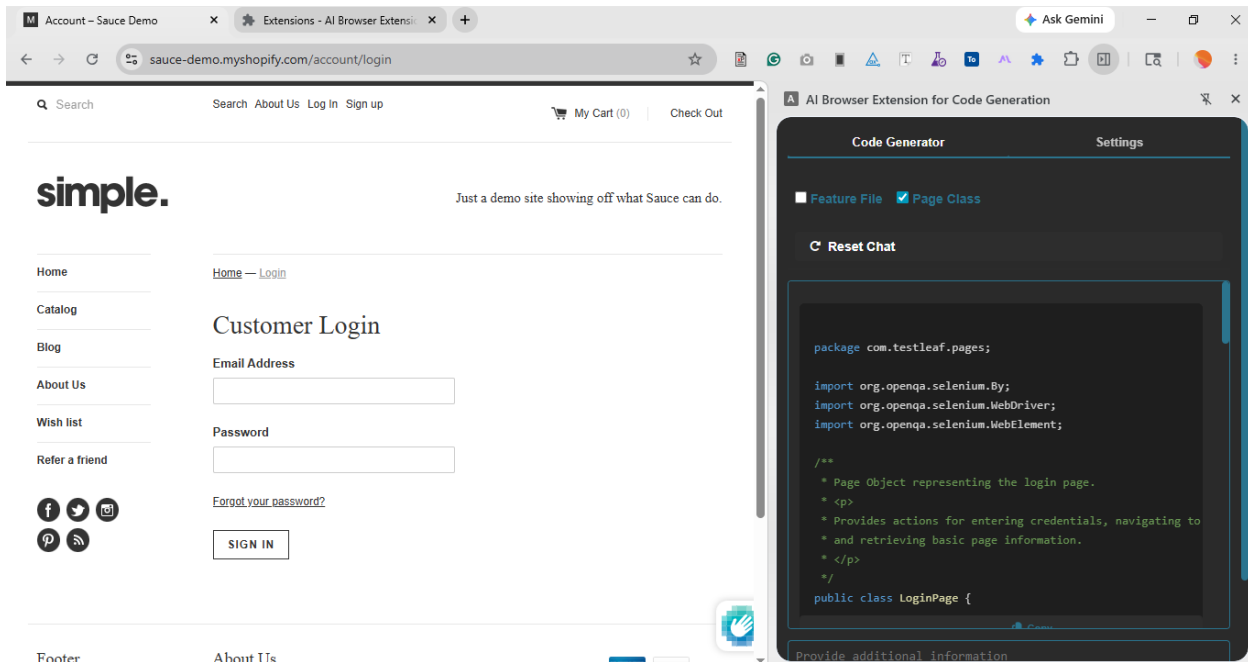
And I click the Continue button

Then I should be redirected to the login page of
"<customDomain>.my.salesforce.com"

Examples:

customDomain	
"mycompany"	
"techsolutions"	

Generate a page class for a webpage



```
package com.testleaf.pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

/**
 * Page Object representing the login page.
 * <p>
 * Provides actions for entering credentials, navigating to related pages,
 * and retrieving basic page information.
 * </p>
 */
public class LoginPage {

    private final WebDriver driver;

    // Locators
    private final By loginLink = By.id("customer_login_link");
    private final By logoLink = By.xpath("//a[contains(@class, 'genai-highlight')] /img");
    private final By emailInput = By.id("customer_email");
    private final By passwordInput = By.id("customer_password");
    private final By forgotPasswordLink = By.xpath("//p[@class='genai-selected'] /a[contains(text(), 'Forgot your password')]");
    private final By signInButton = By.cssSelector("input.button.genai-selected[value='Sign In']");

    /**
     * Constructor.
     *
     * @param driver WebDriver instance to be used by this page object.
     */
    public LoginPage(WebDriver driver) {
```

```

        this.driver = driver;
    }

    /**
     * Enters the e-mail address into the email field.
     *
     * @param email the e-mail address to type.
     */
    public void setEmail(String email) {
        WebElement element = driver.findElement(emailInput);
        element.clear();
        element.sendKeys(email);
    }

    /**
     * Enters the password into the password field.
     *
     * @param password the password to type.
     */
    public void setPassword(String password) {
        WebElement element = driver.findElement(passwordInput);
        element.clear();
        element.sendKeys(password);
    }

    /**
     * Clicks the "Forgot your password?" link.
     */
    public void clickForgotPassword() {
        driver.findElement(forgotPasswordLink).click();
    }

    /**
     * Clicks the "Sign In" button to submit the login form.
     */
    public void clickSignIn() {
        driver.findElement(signInButton).click();
    }

    /**
     * Clicks the "Log In" link (useful when navigating back to the login
page).
     */
    public void clickLoginLink() {
        driver.findElement(loginLink).click();
    }

    /**
     * Returns the source URL of the logo image.
     *
     * @return the value of the {@code src} attribute of the logo image.
     */
    public String getLogoSource() {
        return driver.findElement(logoLink).getAttribute("src");
    }

    /**

```

```
        * Checks whether the logo image is displayed on the page.
        *
        * @return {@code true} if the logo is displayed; {@code false}
otherwise.
        */
    public boolean isLogoDisplayed() {
        return driver.findElement(logoLink).isDisplayed();
    }
}
```